

연구 코드 최적화 및 구조 개선 결과 보고서

과목명: 고급 파이썬 프로그래밍 설계
과제 주제: 코드 개선 및 벤치마킹

제출자: 202650089_한성민

제출 일자: 2026년 6월 21일

1. 대상 연구 코드 소개 및 선정 배경

1.1 실험 맥락 및 역할

선택한 대상 코드는 대규모 자연어 코퍼스 데이터셋을 가공하고 단어 사전(Vocabulary)을 구축한 후, 학습 루프에서 각 문장의 단어 빈도와 속성을 기반으로 의사 모델 가중치(Mock weights)를 반복 업데이트하고, 완료된 모델을 통해 추론(Inference)을 수행하는 자연어 처리(NLP) 연구 파이프라인의 기초 모델입니다.

- 입력: 수십만 라인의 무작위 텍스트 문장 파일 (data_utils/dataset.txt, 50만 라인, 약 39.5 MB)
- 출력: 가공된 최종 어휘 단어 가중치 JSON 파일 (data_utils/model_weights.json)
- 수행 역할: 데이터 전처리 -> 단어 빈도 카운팅을 통한 Vocab 빌드 -> 에폭별 미니배치 학습 루프 -> 가중치 저장 및 추론

1.2 개선 대상 선정 이유

기존 코드는 구현의 편의성에 초점이 맞추어져 있어, 데이터셋의 규모가 수백만 행으로 커지거나 어휘 사전 크기가 상용 수준(수만 개)으로 늘어날 때 심각한 속도 저하 및 메모리 고갈 문제를 야기합니다. 또한, 코드 내부 모듈 간의 결합도가 매우 높아 유지보수와 실험 조건 변경이 어려웠습니다. 이에 따라 성능 최적화와 아키텍처 개선이라는 두 가지 핵심 과제를 수행하기 위해 본 코드를 선정하였습니다.

2. 기존 코드의 문제점 분석

2.1 실행 시간 측면의 문제

DataHandler.load_data_and_build_vocab() 메서드에서 Vocab을 형성할 때, 고유 단어 집합 unique_words에서 앞의 K개 단어에 대해 매번 self.all_words.count(word)를 수행합니다.

- 전체 단어의 총합이 N개(약 500만 개)일 때, 이 연산은 전체 리스트를 K번 풀스캔하게 되므로 시간 복잡도가 매우 큽니다.
- 가상의 어휘 수 K가 커질수록 연산 시간이 선형적으로 배수 증가하는 구조적 한계를 지녔습니다.

2.2 메모리 사용 측면의 문제

f.readlines()를 활용하여 39.5MB 크기의 텍스트 데이터를 통째로 메모리에 로드하고, strip().split()된 모든 단어를 self.all_words 리스트에 평탄화(Flatten)하여 보관합니다. Python 객체 생성 오버헤드와 맞물려 메모리 사용량이 극도로 올라가며, 대용량 파일 가공 시 Out of Memory(OOM)가 발생할 위험이 있습니다.

2.3 자료구조 및 알고리즘의 문제

- 빈도수를 카운트하기 위해 효율적인 해시 테이블 기반 자료구조를 쓰지 않고, 단순 리스트에서의 중복 탐색(list.count)을 행했습니다.
- 배치 학습 시 슬라이싱(self.data_handler.lines[j:i+batch_size])을 매번 호출하여 불필요한 중간 메모리 카피가 반복적으로 이루어졌습니다.

2.4 클래스 구조 결합도 및 SRP 위반

- DataHandler가 model 내부의 딕셔너리에 접근하여 직접 가중치 값을 초깃값으로 변제합니다.
- Trainer 역시 data_handler.lines에 직접 접근하여 슬라이싱하고 model.weights[word] += 0.001과 같이 가중치를ダイ렉트로 변경하여 객체지향 캡슐화를 철저히 위반하고 있습니다.
- NLPModel 내부에는 고정 하이퍼파라미터 설정 정보(batch_size, epochs)와 동적 가중치 런타임 상태가 한데 섞여

있어 단일 책임 원칙(SRP)을 위반합니다.

- Inferencer는 학습 단계의 토큰화 알고리즘(sentence.strip().split())을 개별적으로 중복 구현하여 DRY 원칙을 위반했습니다.

3. 적용한 핵심 수업 개념

1. 시간 복잡도 최적화 & collections.Counter:

- 리스트 풀 스캔 방식을 타파하고 해시 테이블 기반의 collections.Counter를 적용하여 단 1회 순회로 모든 단어 빈도를 수집하고, 힙 정렬 기반의 most_common(k)를 통해 수준으로 Vocab 빌드 단계를 단축시켰습니다.

2. 지연 평가 & Generators:

- yield 키워드를 활용한 제너레이터 파이프라인을 구축하였습니다. 파일 전체를 메모리에 올리지 않고 단어(stream_words) 및 배치(stream_batches)를 스트리밍 방식으로 1줄씩 지연 가공하여 메모리 점유율을 최소화하였습니다.

3. 단일 책임 원칙 및 의존성 주입:

- 역할을 다음과 같이 재배치하여 클래스 간 독립성을 확보했습니다:
- RunConfig (dataclass, slots=True, frozen=True): 순수 하이퍼파라미터 구성 정보 관리.
- TextPreprocessor: 전처리 및 토큰화 표준 로직 정의.
- DataLoader: 데이터 지연 로딩 및 Vocab 생성 전담.
- NLPModel: 모델 가중치의 캡슐화 보존 및 가중치 업데이트 인터페이스 제공.
- Trainer: 배치 이터레이터와 모델 인터페이스를 조율하여 학습 연산 수행.

4. 데코레이터 패턴 & functools.wraps:

- 메인 코드의 비즈니스 로직과 부가적인 메트릭 측정 코드를 완벽하게 물리 분리했습니다.
- timer를 통한 실행 시간 측정 및 memoize를 활용한 추론 시 중복 단어 가중치 조회 연산 캐싱을 적용하였으며, functools.wraps로 원본 메타데이터를 유지했습니다.

4. 개선 과정

4.1 Before vs After 아키텍처 비교

- Before: DataHandler, Trainer, Inferencer가 모두 NLPModel에 직접 묶여서 가중치를 마구 잡이로 변이시켰으며, 파일 전체를 메모리에 적재해 메모리 비대화를 유도함.
- After: 설정(RunConfig), 전처리(TextPreprocessor), 공급(DataLoader), 연산(Trainer/Inferencer)이 철저히 분리되고 상위 클래스의 public method를 통해서만 상호 통신함. 모든 데이터 흐름은 제너레이터를 경유하여 메모리 병목을 원천 봉쇄함.

5. 최적화 결과 및 정량 비교

benchmark/run_benchmark.py를 실행하여 3회 반복 측정 후 얻은 평균 메트릭 데이터셋입니다.

5.1 성능 테이블

버전 (Version)	Vocab 크기 (K)	평균 실행 시간 (초)	픽크 메모리 사용량 (MB)
Baseline (Before)	20	1.5459	434.00
Baseline (Before)	50	1.7506	436.93
Baseline (Before)	100	1.7170	434.96
Baseline (Before)	200	1.7243	434.14
Optimized (After)	20	0.9252	21.95 (95.0% 절감)
Optimized (After)	50	0.9131	22.02 (95.0% 절감)
Optimized (After)	100	0.9205	22.03 (94.9% 절감)
Optimized (After)	200	0.9034	21.86 (95.0% 절감)
Optimized (After)	1000	0.9198	21.96 (대형 스케일 지원)
Optimized (After)	5000	0.9049	21.93 (대형 스케일 지원)

5.2 결과 해석 및 성능 향상 요인

1. 시간 단축 효과:

- 베이스라인은 어휘 크기가 24로 고정된 더미 셋 환경임에도 불필요한 단어 스캔 루프로 인해 약 1.7초 수준의 연산 소요가 지속되었습니다.
- 최적화 코드 적용 결과 전체 소요 시간이 0.9초 대로 수렴하여 약 45~48%의 연산 속도 향상을 달성했습니다.
- 베이스라인과 다르게 힙 정렬 기반 단일 패스 빈도 계산을 하여 K=5000 수준의 대형 보컬 설정에서도 지연 없이 0.90초 만에 학습 파이프라인 전체를 안정적으로 마쳤습니다.

2. 픽크 메모리 점유율 획기적 절감 (95% 감소):

- 파일 전체의 50만 라인을 한꺼번에 String 구조로 로딩하던 기존 베이스라인은 최대 434MB~436MB의 가상 메모리 공간을 차지했습니다.
- Generator 스트리밍(한 줄씩 순차 읽기)을 이관한 최적화 버전은 최대 22MB 수준의 극도로 낮은 메모리로도 데이터 가공 및 트레이닝이 완벽하게 완료되었습니다. 이는 서버급 메모리가 없는 저스펙 엡지 환경이나 초대용량 코퍼스를 다루는 파이프라인에서 필수적인 조치입니다.

3. 가중치 정확도 정합성 통과:

- run_benchmark.py 내의 정합성 검증 모듈을 통해 최적화 전후 생성된 동일 단어 가중치 값의 수치 정밀도를 비교하였고, 가중치 업데이트량이 완벽히 일치하여 최적화 과정에서 로직 상의 어떠한 왜곡도 없음을 확인했습니다.

6. 결론, 트레이드오프 및 한계

6.1 최적화 기법에 따른 트레이드오프

- 지연 평가(Lazy Evaluation)의 입출력 병목:
제너레이터 스트리밍은 메모리 사용량을 대폭 절감하지만, 매 에폭마다 동일한 대용량 텍스트 파일을 디스크에서 다시 한 줄씩 읽는 디스크 I/O가 재발합니다. 메모리에 전부 올리는 방식은 I/O가 1번만 발생하는 반면, 지연 평가 방식은 디스크 성능에 종속됩니다.
만약 시스템 메모리가 매우 넉넉하고 디스크가 HDD 수준으로 느리다면, 가공 단계에서 캐싱 혹은 온메모리 가공을

타협하여 적용할 수 있습니다.

6.2 향후 개선 방안

- 멀티프로세싱 및 병렬화:

파이썬의 GIL(Global Interpreter Lock)로 인해 싱글 스레드로 데이터 전처리를 하고 있습니다. multiprocessing 패키지를 사용해 파일을 분할하여 병렬 전처리하고 이를 공유 큐에 넣어 처리하면 초대용량 실험 코드의 추가적인 속도 배가 효과를 기대할 수 있습니다.

- 바이너리 직렬화:

모델 가중치를 텍스트 형식인 JSON 대신 binary 계열 포맷(Protocol Buffers, MessagePack, PyTorch safetensors 등)으로 저장 및 복원하면 파일 크기 단축 및 입출력 직렬화 비용을 크게 낮출 수 있습니다.